



400-650-7088

Tong**NCS** V2.0

Spring 开发手册

版本：2043A01

2025年09月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

文档版本	适用产品版本	更新内容	更新日期
2043A01	2.0.4.3	修改“声明”中的版权信息。	2025-09
2042_P1A01	2.0.4.2_P1	无	2025-08
2042A01	2.0.4.2	修改如下章节： • 启动配置管理 & 启动服务管理，补充获取 password 字段密码的路径。 • 启用 TongNCS，修改 password 字段值。	2025-06
2041A01	2.0.4.1	无	2024-12
2040A01	2.0.4.0	修改如下章节： • 导入 jar 文件，将手动复制 jar 到指定目录修改为运行“install_maven_jar.sh”脚本导入 jar 文件。 • 配置 pom.xml 文件，将“com.tong.ncs”修改为“com.tongtech.tongncs”。	2024-10
2030A01	2.0.3.0	无	2024-04
2020A01	2.0.2.0	无	2023-12
2010A01	2.0.1.0	首次发布。	2023-10

前言

本文档是 TongNCS 产品的用户手册之一，详细介绍 Java 研发人员使用 TongNCS 提供的 Spring 开发工具包进行开发。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

- 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

- 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

- 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

- Linux 操作系统基本操作；
- 熟悉 TongNCS 的运维管理；
- 熟悉 TongNCS 运行的配套软件，如数据库。

- 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

- 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户手册集

TongNCS 为您提供如下用户使用手册。

编号	用户手册	说明
1	001_TongNCS_V2.0产品安装手册	详细介绍了在单机环境、Docker 环境以及 K8s 环境下快速安装 TongNCS。
2	002_TongNCS_V2.0产品使用手册	详细介绍了 TongNCS 详细功能使用说明。

编号	用户手册	说明
3	003_TongNCS_V2.0Java Client 开发手册	详细介绍 Java 研发人员使用 TongNCS 提供的 Java Client 开发工具包进行开发。
4	004_TongNCS_V2.0Spring 开发手册	详细介绍 Java 研发人员使用 TongNCS 提供的 Spring 开发工具包进行开发。
5	005_TongNCS_V2.0SpringBoot 开发手册	详细介绍 Java 研发人员使用 TongNCS 提供的 SpringBoot 开发工具包进行开发。
6	006_TongNCS_V2.0SpringCloud 开发手册	详细介绍 Java 研发人员使用 TongNCS 提供的 SpringCloud 开发工具包进行开发。
7	007_TongNCS_V2.0OpenAPI 使用手册	详细介绍 Java 研发人员使用 TongNCS 提供的 OpenAPI 开发工具进行开发。
8	008_TongNCS_V2.0OpenLDAP 认证使用手册	详细介绍 TongNCS 如何使用 OpenLDAP 进行用户认证以及 OpenLDAP 的安装与使用。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

- 您的姓名
- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	3
1. 使用前导读	6
2. 前置准备	7
3. 开发步骤	8
3.1. 创建 Maven 工程	8
3.2. 导入jar文件	8
3.3. 配置 pom.xml 文件	8
3.4. 启动配置管理	8
3.5. 发布配置	8
3.6. 查询配置	9
3.7. 监听配置的变化	9
3.8. 启用服务管理	9
3.9. 获取 NamingService 实例	9
3.10. 注册服务实例	9
3.11. 列出所有服务实例	10
3.12. 监听服务信息变化	10
4. Spring 注解参考	11
4.1. 启用	
TongNCS (@EnableTongNCS、@EnableTongNCSDiscovery、@EnableTongNCSConfig)	11
4.2. TongNCS 属性 (@TongNCSProperties)	11
4.3. 外部配置	12
4.3.1. 定义配置读取位置 (@PropertySource、@PropertySources)	12
4.3.2. 获取单个配置 (@TongNCSValue)	13
4.3.3. 获取配置至	
Bean (@TongNCSConfigurationProperties、@TongNCSPROPERTY、@TongNCSIgnore)	13
4.4. 依赖注入 @TongNCSInjected	13
4.5. 配置监听 @TongNCSConfigListener	14

1. 使用前导读

本文档用于指导 Java 研发人员使用 TongNCS 提供的 Spring 开发工具包进行开发。

2. 前置准备

- 完成 TongNCS V2.0 产品安装并启动服务。
- TongNCS 已启用认证，用户名/密码：tongncs/Admin123#

3. 开发步骤

3.1. 创建 Maven 工程

创建 Spring 类型的 Maven 工程。

3.2. 导入jar文件

运行“sdk/java/install_maven_jar.sh”，将 sdk 的 jar 文件复制至本地 maven 仓库。

如需发布至远程自建 maven 仓库，则运行“sdk/java/deploy_maven_jar.sh”。

3.3. 配置 pom.xml 文件

在 <dependencies> 中，加入如下依赖。

```
...
<dependency>
  <groupId>com.tongtech.tongncs</groupId>
  <artifactId>tongncs-spring-context</artifactId>
  <version>2.0.4</version>
</dependency>
...
```

3.4. 启动配置管理

```
@Configuration
@EnableTongNCSCfg(globalProperties = @TongNCSProperties(serverAddr = "127.0.0.1:8848", username =
"tongncs", password = "c9053c27545d49e933bf9e228e95ac7e"))
@TongNCSPROPERTYSource(dataId = "tongncs-samples-config.yaml", autoRefreshed = true)
public class TongNCSCfgConfiguration {
```

添加 @EnableTongNCSCfg 注解启用 TongNCS Spring 的配置管理服务，通过属性标定 TongNCS 服务器地址与用户信息。

注意：其中 password 字段不是管控台密码，需要登录管控台，点击【系统管理】-【认证授权】-【复制秘钥】进行获取。

使用 @TongNCSPROPERTYSource 注解加载指定 dataId 的配置源，autoRefreshed 表示开启自动更新。

3.5. 发布配置

```
@TongNCSInjected
```

```
private ConfigService configService;
configService.publishConfig(dataId, group, content);
```

使用 @TongNCSInjected 注解注入 TongNCS 的 ConfigService 实例，调用 publishConfig 方法发布配置。

3.6. 查询配置

```
@TongNCSValue(value = "${name:Tom}", autoRefreshed = true)
private String name;
```

通过 @TongNCSValue 注解查询配置并设置到绑定变量，autoRefreshed 表明是否自动更新变量。

也可以通过 Spring 的 @Value 注解与 @RefreshScope 注解完成相同功能。

3.7. 监听配置的变化

```
@TongNCSConfigListener(dataId = "TongNCS-Listener-Demo.yaml", groupId = "DEFAULT_GROUP")
public void onReceived(String context){
    System.out.println(context);
}
```

通过 @TongNCSConfigListener 注解注入配置变更监听方法，配置发生变更时，此方法被调用。

3.8. 启用服务管理

```
@Configuration
@EnableTongNCSDiscovery(globalProperties = @TongNCSProperties(serverAddr = "127.0.0.1:8848", username =
"tongncs", password = "c9053c27545d49e933bf9e228e95ac7e"))
public class TongNCSNamingConfiguration {
}
```

注意：

其中 password 字段不是管控台密码，需要登陆管控台，点击【系统管理】-【认证授权】-【复制秘钥】进行获取。

3.9. 获取 NamingService 实例

```
@TongNCSInjected
private NamingService namingService;
```

3.10. 注册服务实例

```
namingService.registerInstance(SERVER_NAME, GROUP, "10.10.10.1", 9001)
```

3.11. 列出所有服务实例

```
List<Instance> instances = namingService.getAllInstances(SERVER_NAME);
for (Instance instance : instances) {
    System.out.println(instance.getIp() + ":" + instance.getPort());
}
```

3.12. 监听服务信息变化

```
namingService.subscribe(SERVER_NAME, new AbstractEventListener() {
    @Override
    public Executor getExecutor() {
        return executor;
    }

    @Override
    public void onEvent(Event event) {
        System.out.println("serviceName: " + ((NamingEvent) event).getServiceName());
        System.out.println("instances from event: " + ((NamingEvent) event).getInstances());
    }
});
```

4. Spring 注解参考

4.1. 启用 TongNCS

(@EnableTongNCS、@EnableTongNCSDiscovery、@EnableTongNCSConfig)

@EnableTongNCSDiscovery 启用服务发现功能。

```
@EnableTongNCSDiscovery(globalProperties = @TongNCSProperties(serverAddr = "127.0.0.1:8848", username = "tongncs", password = "c9053c27545d49e933bf9e228e95ac7e"))
```

@EnableTongNCSConfig 启用配置管理功能。

```
@EnableTongNCSConfig(globalProperties = @TongNCSProperties(serverAddr = "127.0.0.1:8848", username = "tongncs", password = "c9053c27545d49e933bf9e228e95ac7e"))
```

@EnableTongNCS 综合上述两者启用所有 TongNCS 功能。

```
@EnableTongNCS(globalProperties = @TongNCSProperties(serverAddr = "127.0.0.1:8848", username = "tongncs", password = "c9053c27545d49e933bf9e228e95ac7e"))
```

globalProperties 是任何 @EnableTongNCS、@EnableTongNCSDiscovery 或 @EnableTongNCSConfig 中的必选属性，其类型为 @TongNCSProperties。

globalProperties 属性，详见 [TongNCS属性](#) 部分。

4.2. TongNCS 属性 (@TongNCSProperties)

TongNCS 属性分为全局属性 globalProperties 与自定义属性两类 properties，类型都为 @TongNCSProperties。

- globalProperties

globalProperties 是 @EnableTongNCS、@EnableTongNCSDiscovery、@EnableTongNCSConfig 注解的必选属性，定义全局和默认属性，影响到所有其他相关注解，

如@TongNCSInjected、@TongNCSConfigListener、@TongNCSPropertySource、@TongNCSConfigurationProperty 等，它设置为具有最低优先级，可以被自定义属性覆盖。

- properties:

properties 是

@TongNCSInjected、@TongNCSConfigListener、@TongNCSPropertySource、@TongNCSConfigurationProperty等注解的可选属性，定义该注解的自定义属性，优先级高于全局属性，范围仅为当前注解。

@TongNCSProperties 是全局和自定义 TongNCS 属性的统一注解，支持占位符，下表列举出 @TongNCSProperties 注解中定义的属性以及它与属性、占位符的对应关系，如下表所示。

Attribute	Property	Placeholder	Required
endpoint()	endpoint	\${tongncs.endpoint:}	N
namespace()	namespace	\${tongncs.namespace:}	N
accessKey()	access-key	\${tongncs.access-key:}	N
secretKey()	secret-key	\${tongncs.secret-key:}	N
serverAddr()	server-addr	\${tongncs.server-addr:}	Y
contextPath()	context-path	\${tongncs.context-path:}	N
clusterName()	cluster-name	\${tongncs.cluster-name:}	N
encode()	encode	\${tongncs.encode:UTF-8}	N
username()	username	\${tongncs.username:}	N
password()	password	\${tongncs.password:}	N

4.3. 外部配置

外部化配置允许应用程序接收外部属性源以控制运行时行为。

TongNCS 在应用程序外部运行单独的进程以维护应用程序配置。TongNCS spring 开发包提供了对象绑定，动态配置（自动刷新）等功能。

4.3.1. 定义配置读取位置（@PropertySource、@PropertySources）

@PropertySource 注解用于指定配置服务所处地址信息。

```
@TongNCSPropertySource(dataId = "tongncs-samples-config.yaml", autoRefreshed = true)
public class TongNCSConfigConfiguration {}
```

@TongNCSPropertySources 注解可定义多个配置服务来源，是多个 @PropertySource 的集合。

```
@TongNCSPropertySources({
    @TongNCSPropertySource(dataId = "app.properties", groupId = "multi-data-ids", autoRefreshed = true),
    @TongNCSPropertySource(dataId = "redis.properties", groupId = "multi-data-ids")
})
```

4.3.2. 获取单个配置 (@TongNCSValue)

@TongNCSValue 注解用于获取 TongNCS 中的配置项，并绑定到 Java 类的属性中。

通过 @TongNCSValue 注解的 value 属性设置待获取的属性，autoRefreshed 属性表明是否动态更新配置。

```
@TongNCSValue(value = "${name:Tom}", autoRefreshed = true)
private String name;
```

4.3.3. 获取配置至 Bean

(@TongNCSConfigurationProperties、@TongNCSProperty、@TongNCSIgnore)

通过在 Bean 上添加 @TongNCSConfigurationProperties 注解，指定好配置文件的前缀，配置数据自动填充到 Bean 中，@TongNCSProperty 通过 value 属性匹配属性名，@TongNCSIgnore 则忽略其修饰的属性。

```
@TongNCSConfigurationProperties(dataId = "TongNCSConfigPojo.yaml", groupId = "spring-example",
autoRefreshed = true)
public class TongNCSConfigPojo {

    @TongNCSProperty(value = "name")
    private String name;

    @TongNCSProperty(value = "age")
    private int age;

    @TongNCSIgnore
    private String sex;
}
```

4.4. 依赖注入 @TongNCSInjected

@TongNCSInjected 是一个核心注解，用于在 Spring Beans 中注入 ConfigService 或 NamingService 实例，并使这些实例可缓存。

这意味着若 @TongNCSProperties 相等，则实例将是相同的，无论属性是来自全局还是自定义的 TongNCS 属性。

```
@TongNCSInjected
private ConfigService configService;
@TongNCSInjected(properties = @TongNCSProperties(encode = "UTF-8"))
private ConfigService configService2;
@TongNCSInjected(properties = @TongNCSProperties(encode = "GBK"))
```

```
private ConfigService configService3;  
@TongNCSInjected  
NamingService namingService;  
@TongNCSInjected(properties = @TongNCSProperties(encode = "UTF-8"))  
private NamingService namingService2;
```

4.5. 配置监听 @TongNCSTConfigListener

添加一个监听器，它将监听配置的变化。可以通过在 Spring Bean 中添加配置变更监听器方法来执行此操作。

```
@TongNCSTConfigListener(dataId = "TongNCS-Listener-Demo.yaml", groupId = "DEFAULT_GROUP")  
public void onReceived(String context){  
    System.out.println(context);  
}
```

